

MỤC LỤC

HTML CSS

Linh tinh.....	3
Tự tạo biến khi đặt id hoặc name cho một input/output trong form:....	3
Chiều cao ban đầu.....	3
Thẻ trong thẻ.....	3
Pseudo-classes (Giả lớp trong CSS).....	3
Pseudo-elements selectors (Bộ chọn phần tử giả).....	4
Display.....	5
flex.....	6
Visibility.....	8
Text-align và align-items.....	9
Thẻ span.....	9
Table.....	10
Overflow.....	11
Position.....	12
Cursor.....	13
Transition.....	14
transform.....	15
label.....	16
name.....	17
nav.....	18
box-shadow.....	19
Text-decorating.....	19
Counter.....	20
flex-wrap.....	21
aspect-ratio.....	22
border-radius.....	22
gap.....	23
Table.....	24
Background-image.....	26
1. background-image.....	26
✓ 2. background-repeat.....	26
✓ 3. background-size.....	26
✓ 4. background-position.....	26

✓ 5. background-attachment.....	27
✓ 6. background-color.....	27
✓ 7. background (thu gọn).....	27
data-*	27
 opacity là gì?.....	28
BOOTSTRAP	
Color.....	29
Grid.....	30
Display.....	31
Typography.....	31
Table.....	31
Image.....	32
Aligning Images.....	32
Centered Image.....	32
.....	32
Alert.....	32
Button.....	32
Button-group (inline).....	33
Progress bar.....	34
Spinners.....	34
Pagination (chuyển trang).....	35
List group.....	35
Badge.....	36
Data-bs-toggle.....	37
Dropdown.....	38
Collapse.....	38
aria-expanded.....	39
aria-controls.....	39
margin.....	40
Forms.....	40
Check.....	41
Radio.....	42
Toggle Switches.....	42
Card.....	42
JS	
element.classList.....	43

 localStorage là gì?.....	45
--	----

JQUERY

1. jQuery - Lý thuyết cơ bản.....	47
1.1. jQuery là gì?.....	47
1.2. Cách sử dụng jQuery.....	47
1.3. Selector (Chọn phần tử).....	48
1.4. DOM Manipulation (Thao tác DOM).....	48
1.5. Xử lý sự kiện (Event Handling).....	48
1.6. Hiệu ứng (Effects).....	49
1.7. jQuery Chaining.....	49
1.8. Document Ready.....	49

AJAX

2. Ajax - Lý thuyết cơ bản.....	49
2.1. Ajax là gì?.....	50
2.2. Các lợi ích của Ajax.....	50
2.3. Cách sử dụng Ajax với jQuery.....	50
2.3.1. .ajax().....	50
2.3.2. Các phương thức tiện ích khác.....	50
2.4. Các tham số quan trọng của Ajax.....	51
2.5. Xử lý dữ liệu trả về.....	51
2.6. Bất đồng bộ (Asynchronous).....	51
3. Tổng kết.....	52

YUP

 1. Kiểm tra với string() – Chuỗi.....	52
 2. Kiểm tra với number() – Số.....	53
 3. Kiểm tra với date() – Ngày tháng.....	53

cài project react: `npx create-react-app`

cài react hook form + yup: `npm install react-hook-form @hookform/resolvers yup`

rafce

HTML CSS

Linh tinh

Tự tạo biến khi đặt id hoặc name cho một input/output trong form:

Thực ra trong HTML **form**, khi bạn đặt id hoặc name cho một input/output, trình duyệt tự động tạo một biến cùng tên cho bạn trong JavaScript lúc đó.

Chiều cao ban đầu

Theo CSS, nếu bạn không khai báo height cho một phần tử (như <html> hoặc <body>), thì nó **mặc định là auto**, tức cao bằng nội dung bên trong.

Thẻ trong thẻ

vd: thẻ img, span trong thẻ a

Khi bạn đặt **** và **** bên trong thẻ **<a>**, nghĩa là bạn đang làm cho cả logo (ảnh) và chữ trở thành một liên kết duy nhất.

Pseudo-classes (Giả lớp trong CSS)

Định nghĩa: **Pseudo-class** là một từ khóa thêm vào selector để **chọn phần tử ở trạng thái đặc biệt**, mà không cần thay đổi HTML.

Cú pháp: `selector:pseudo-class { CSS properties }`

Tên	Ý nghĩa
<code>:hover</code>	Khi người dùng rê chuột lên phần tử
<code>:active</code>	Khi phần tử đang được click
<code>:focus</code>	Khi phần tử nhận được focus (ví dụ: input đang được gõ)
<code>:visited</code>	Link đã được truy cập
<code>:link</code>	Link chưa được truy cập
<code>:first-child</code>	Phần tử con đầu tiên của parent
<code>:last-child</code>	Phần tử con cuối cùng của parent
<code>:nth-child(n)</code>	lấy ra phần tử con theo tiêu chí n (vd: even chẵn, odd lẻ)
<code>:checked</code>	Checkbox hoặc radio được chọn
<code>:disabled</code>	Input/button bị khóa
<code>:enabled</code>	Input/button có thể tương tác
<code>:not(selector)</code>	Chọn phần tử không phải selector đó

Pseudo-elements selectors (Bộ chọn phần tử giả)

Dùng để **chọn và định dạng một phần cụ thể bên trong phần tử HTML**, như chữ cái đầu tiên, dòng đầu tiên hoặc thêm nội dung trước/sau phần tử.

Cú pháp: `selector::pseudo-element`

Pseudo-element	Mô tả	Ví dụ
::before	Thêm nội dung trước phần tử	<code>p::before { content: "👉 "; }</code>
::after	Thêm nội dung sau phần tử	<code>p::after { content: " ✓ "; }</code>
::first-letter	Chọn chữ cái đầu tiên của đoạn văn	<code>p::first-letter { font-size: 200%; color: red; }</code>
::first-line	Chọn dòng đầu tiên của đoạn văn	<code>p::first-line { font-weight: bold; }</code>
::selection	Chọn phần văn bản được highlight (bôi đen) bởi người dùng	<code>p::selection { background: yellow; }</code>

Display

Công dụng: Xác định **cách phần tử xuất hiện** trên trang.

Giá trị display	Ý nghĩa	Ghi chú
block	Hiển thị như một khối (chiếm toàn bộ chiều ngang)	Ví dụ: <div>, <p>
inline	Hiển thị trên cùng 1 dòng, chỉ chiếm đủ nội dung	Ví dụ: , <a>
inline-block	Vừa inline (trên cùng dòng), vừa set được width/height như block	Thường dùng làm menu, button
flex	Kích hoạt mô hình Flexbox (chia layout linh hoạt). Các phần tử bên trong nó (các "item") sẽ xếp hàng ngang (theo mặc định) và dễ dàng canh chỉnh vị trí, khoảng cách, kích thước.	Quản lý các item con rất mạnh
grid	Kích hoạt mô hình Grid (bố trí lưới)	Chia ô, chia dòng, cực kỳ chuyên nghiệp
none	Ẩn phần tử hoàn toàn, không chiếm chỗ	Giống như xóa khỏi trang

flex

Cú pháp: flex: [flex-grow] [flex-shrink] [flex-basis];

1. flex-grow

👉 Ý nghĩa:

- Xác định **phần tử có được giãn ra** để chiếm không gian dư hay không.

- Giá trị là **số không âm**.

✅ Giá trị:

- 0 (mặc định): không giãn, giữ nguyên kích thước ban đầu.
- 1 trở lên: cho phép giãn ra. Các phần tử sẽ chia phần dư **theo tỷ lệ**.

🔧 Ví dụ:

`flex-grow: 2;`

→ phần tử này sẽ giãn **gấp đôi** phần tử có `flex-grow: 1`.

📦 2. **flex-shrink**

👉 Ý nghĩa:

- Xác định phần tử **co lại bao nhiêu** khi không gian bị **thu hẹp**.
- Giá trị là **số không âm**.

✅ Giá trị:

- 0: không co lại khi thiếu chỗ (có thể tràn dòng).
- 1 (mặc định): cho phép co lại.
- 2, 3, ...: co lại **nhANH hơn**, tỷ lệ cao hơn.

🔧 Ví dụ:

`flex-shrink: 0;`

→ giữ nguyên kích thước, không co dù không gian bị thiếu.

📏 3. **flex-basis**

👉 Ý nghĩa:

- Xác định **kích thước khởi đầu** của phần tử trước khi xét đến grow hay shrink.

✅ Giá trị:

- auto: dùng width hoặc height nếu có (mặc định).
- 0: bỏ qua kích thước ban đầu, bắt đầu từ 0.
- Đơn vị cố định: 100px, 30%, 10rem...

 Ví dụ:

- flex-basis: 200px;

→ phần tử sẽ **cố gắng chiếm 200px** ban đầu, rồi mới tính tới co/giãn.

 Một số giá trị thường dùng của flex

Viết tắt	Ý nghĩa đầy đủ	Mô tả nhanh
flex: 1	1 1 0%	Chia đều không gian còn trống
flex: 2	2 1 0%	Gấp đôi không gian so với flex: 1
flex: 0	0 1 0%	Không giãn ra, chỉ co lại nếu cần
flex: auto	1 1 auto	Linh hoạt co/giãn dựa trên kích thước nội dung
flex: none	0 0 auto	Không giãn, không co, dùng kích thước mặc định

Visibility

Công dụng: Điều khiển phần tử **có thấy được** hay **ẩn đi**, nhưng **vẫn chiếm chỗ** trên giao diện.

Giá trị visibility	Ý nghĩa	Ghi chú

visible	Hiển thị phần tử (mặc định)	Phần tử bình thường hiện ra
hidden	Ẩn phần tử, nhưng vẫn chiếm chỗ	Dùng khi muốn "giấu" nhưng không thay đổi bố cục
collapse	(ít dùng) Ẩn các dòng/ô của table	Chủ yếu áp dụng cho table-row hoặc table-column

Text-align và align-items

Thuộc tính	Thuộc nhóm	Tác động lên	Ý nghĩa	Giá trị phổ biến
text-align	Thuộc tính căn chỉnh nội dung văn bản	Nội dung bên trong phần tử block (ví dụ: <div>, <p>)	Căn nội dung chữ theo trái, phải, giữa, justify	left, right, center, justify
align-items	Thuộc tính của Flexbox hoặc Grid	Các phần tử con trong flex container	Căn các item theo chiều dọc trong container	flex-start, flex-end, center, stretch, baseline

Thẻ span

✓ Định nghĩa:

 là **thẻ inline** (không xuống dòng), dùng để **bao quanh một phần nhỏ của văn bản** hoặc nội dung để **áp dụng CSS hoặc JavaScript riêng biệt** cho phần đó.

✓ Không có ý nghĩa nội dung (semantic) – chỉ để nhóm và định dạng.

📌 Ví dụ sử dụng với CSS:

```
<p>Tôi thích <span class="highlight">phô mai</span>
và <span class="highlight">sô cô la</span>.</p>

<style>
  .highlight {
    color: orange;
    font-weight: bold;
  }
</style>
```

👉 Kết quả: chữ **"phô mai"** và **"sô cô la"** sẽ có màu cam và đậm.

⚠ Khi nào dùng ?

- Khi bạn **muốn định dạng một phần nhỏ** trong dòng văn bản (ví dụ đổi màu, đổi font).
- Khi bạn **không cần tạo khối** như <div> mà vẫn muốn gán class, id hoặc xử lý JavaScript.

Table

```
<table>
  <tr>
    <th>Company</th>
```

```

    <th>Contact</th>
    <th>Country</th>
</tr>
<tr>
    <td>Alfreds Futterkiste</td>
    <td>Maria Anders</td>
    <td>Germany</td>
</tr>
<tr>
    <td>Centro comercial Moctezuma</td>
    <td>Francisco Chang</td>
    <td>Mexico</td>
</tr>
</table>

```

Tên thuộc tính	Ý nghĩa	Ví dụ
border-collapse	cách hiển thị đường viền (border) giữa các ô trong table collapse: 1 viền separate: 2 viền	table { border-collapse: collapse separate; }
text-align		
colspan	To make a cell span over multiple columns, use the colspan attribute	
rowspan	To make a cell span over multiple rows, use the rowspan attribute	

Overflow

Thuộc tính `overflow` kiểm soát **cách hiển thị nội dung khi nó vượt ra ngoài kích thước của phần tử chứa nó** (thường dùng với `div` có chiều cao hoặc chiều rộng cố định).

overflow-x: visible | hidden | scroll | auto;

Giá trị	Ý nghĩa	Mô tả
visible	Hiển thị tràn ra ngoài	(mặc định) – nội dung vượt ra sẽ vẫn hiện
hidden	Ẩn phần tràn ra	Không cho thấy nội dung vượt giới hạn
scroll	Hiện thanh cuộn luôn	Dù có tràn hay không
auto	Hiện thanh cuộn khi cần	Chỉ hiện nếu nội dung bị

Position

Giá trị	Ý nghĩa
static (default)	Static positioned elements are not affected by the top, bottom, left, and right properties. is not positioned in any special way; it is always positioned according to the normal flow of the page
relative	is positioned relative to its normal position. Setting the top, right, bottom, and left properties of a relatively-positioned element will cause it to be adjusted away from its normal position
fixed	is positioned relative to the viewport, which means it always stays in the same place even if the page is scrolled. The top, right, bottom, and left properties are used to position the element.
absolute	An element with position: absolute; is positioned relative

	to the nearest positioned ancestor (instead of positioned relative to the viewport, like fixed). However; if an absolute positioned element has no positioned ancestors, it uses the document body, and moves along with page scrolling. (Tuyệt đối so với phần tử chứa gần nhất có position: relative/absolute/fixed) Note: Absolute positioned elements are removed from the normal flow, and can overlap elements.
sticky	is positioned based on the user's scroll position. (dính vào 1 vị trí khi cuộn) A sticky element toggles between relative and fixed, depending on the scroll position. It is positioned relative until a given offset position is met in the viewport - then it "sticks" in place (like position:fixed).

Cursor

✓ Dùng để điều khiển **hình dạng con trỏ** khi rê chuột vào phần tử.

```
button {
  cursor: pointer;
}
```

Giá trị	Ý nghĩa
default	Mũi tên bình thường
pointer	Bàn tay (thường dùng cho nút bấm)
text	Dạng con trỏ khi rê vào đoạn văn bản

move	Dấu mũi tên 4 chiều (kéo được)
not-allowed	Biểu tượng cấm
wait	Đồng hồ chờ

Transition

✅ Dùng để tạo hiệu ứng chuyển động mượt khi thay đổi thuộc tính (như màu, kích thước, vị trí...).

✨ Ví dụ:

```
.box {
  width: 100px;
  height: 100px;
  background: red;
  transition: background 0.3s ease;
}

.box:hover {
  background: blue;
}

<!-- Hoặc -->
transition: all 1s ease-in;
```

👉 Khi bạn rê chuột vào `.box`, màu chuyển từ đỏ sang xanh mượt mà trong 0.3 giây.

 **Cú pháp tổng quát:**

```
transition: [thuộc_tính] [thời_gian]
[kiểu_chuyển_động];
```

Kiểu chuyển động

Giá trị	Miêu tả
ease	Bắt đầu chậm, tăng tốc, rồi chậm lại (mặc định)
linear	Tốc độ đều đặn từ đầu đến cuối
ease-in	Bắt đầu chậm rồi tăng tốc dần
ease-out	Bắt đầu nhanh rồi chậm lại
ease-in-out	Bắt đầu và kết thúc chậm, giữa nhanh

transform

transform là một thuộc tính CSS dùng để **biến đổi hình dạng hoặc vị trí** của phần tử HTML. Bạn có thể **xoay, phóng to, thu nhỏ, di chuyển**, hoặc **ngiêng** phần tử theo nhiều cách khác nhau.

♦ Các giá trị thường dùng của transform:

Hàm	Ý nghĩa
translate(x, y)	Di chuyển phần tử theo trục X và Y
scale(x, y)	Phóng to/thu nhỏ phần tử

<code>rotate(deg)</code>	Xoay phần tử theo góc (ví dụ: 45deg)
<code>skew(x, y)</code>	Nghiêng phần tử theo trục X và Y
<code>matrix(...)</code>	Biến đổi nâng cao (hiếm dùng trực tiếp)

♦ Ví dụ:

```
.box {
  transform: translateX(50px); /* Di chuyển sang
phải 50px */
}
```

```
.box:hover {
  transform: rotate(15deg); /* Khi hover thì xoay
15 độ */
}
```

`translate(-50%)` thường được dùng trong CSS để di chuyển phần tử ngược lại 50% chính kích thước của

Bạn cũng có thể kết hợp nhiều transform:

```
transform: translate(20px, 10px) scale(1.2)
rotate(10deg);
```

label

1. Mục đích của for

- for **liên kết** `<label>` với một **phần tử form** (thường là `<input>`, `<select>`, `<textarea>`).

Giá trị của **for** **phải** trùng với **id** của phần tử đó.

Ví dụ:

```
<label for="username">Tên đăng nhập:</label>  
<input type="text" id="username" name="username">
```

– Khi bạn **click vào chữ “Tên đăng nhập:”**, con trỏ sẽ tự động **chuyển vào ô input** có **id="username"**.

2. Lợi ích khi dùng for

1. Tăng khả năng truy cập (accessibility)

– Người dùng dùng bộ đọc màn hình (screen reader) sẽ dễ dàng biết label nào hướng dẫn cho ô nào.

2. Mở rộng vùng nhấn

– Thay vì phải click đúng lên ô input nhỏ, bạn có thể click lên label to hơn để tập trung ô input.

3. Thân thiện với thiết bị di động

– Vùng nhấn lớn giúp người dùng chạm chính xác hơn trên điện thoại, máy tính bảng.

3. Khi nào không cần for

Nếu bạn **bọc trực tiếp** input bên trong label thì không cần for, vì chúng đã liên kết sẵn:

```
<label>  
  Tên đăng nhập:  
  <input type="text" name="username">  
</label>
```

- Cách này cũng cho phép click vào chữ label để focus vào input.

name

vd trong input

1. Mục đích của name trong <input>

- Thuộc tính name trong thẻ <input> giúp **đặt tên** cho dữ liệu mà người dùng nhập vào, và khi gửi form đi, **dữ liệu đó sẽ được gửi với tên này**.
- Khi bạn gửi form tới server (thông qua GET hoặc POST), các giá trị trong form sẽ được gán cho các trường dữ liệu theo tên của name và gửi dưới dạng key=value

2. Tại sao cần name?

- **Xác định dữ liệu:** name là cách duy nhất để server nhận biết giá trị nào tương ứng với mỗi trường trong form.
- **Đồng nhất:** Nếu bạn không sử dụng name, dữ liệu từ form sẽ không được gửi đúng cách hoặc không gửi được tới server.
- **Xử lý dễ dàng:** Khi server nhận được dữ liệu, các trường sẽ được phân biệt và có thể xử lý riêng biệt (ví dụ như lưu vào cơ sở dữ liệu).

nav

"Thẻ nav" trong HTML là một **thẻ ngữ nghĩa (semantic tag)** dùng để định nghĩa **khu vực điều hướng (navigation)** trên trang web, ví dụ như menu chính, liên kết đến các trang khác, thanh điều hướng dọc hoặc ngang.

Tính năng chính:

- Dùng để chứa các liên kết điều hướng chính trong trang.

- Giúp các trình đọc màn hình (screen readers) và công cụ tìm kiếm hiểu rõ cấu trúc trang web.
- Có thể dùng nhiều thẻ <nav> trong một trang (ví dụ: một cho menu chính, một cho chân trang).

box-shadow

box-shadow – một thuộc tính CSS dùng để **tạo bóng đổ** cho phần tử, giúp giao diện trông nổi bật và có chiều sâu hơn.



Cú pháp cơ bản:

```
box-shadow: <x-offset> <y-offset> <blur-radius>
<spread-radius> <color>;
```

Trong đó:

- x-offset: độ lệch bóng theo **trục ngang** (px).
- y-offset: độ lệch bóng theo **trục dọc** (px).
- blur-radius: độ **mờ** của bóng (càng lớn càng mờ).
- spread-radius: độ **lan rộng** của bóng (tùy chọn).
- color: màu của bóng.

```
box-shadow: 0 4px 8px rgba(0, 0, 0, 0.2); /* đổ bóng mềm */
```

Text-decorating

text-decoration là **thuộc tính CSS dùng để trang trí văn bản**, ví dụ như gạch chân, gạch ngang, gạch trên, hoặc bỏ gạch chân mặc định của liên kết.



Các giá trị thường dùng:

Giá trị	Ý nghĩa
---------	---------

none	Không có trang trí (bỏ gạch chân, v.v.)
underline	Gạch chân dưới văn bản
overline	Gạch ngang trên văn bản
line-through	Gạch ngang xuyên giữa văn bản
underline overline	Gạch cả trên lẫn dưới

Counter

 Tổng quan nhanh:

Thuộc tính / Hàm	Chức năng
counter-reset	Tạo mới hoặc đặt lại giá trị một bộ đếm về 0 (hoặc giá trị cụ thể).
counter-increment	Tăng giá trị bộ đếm mỗi khi phần tử xuất hiện.
content	Chèn nội dung giả lập (dùng trong ::before hoặc ::after).
counter() / counters()	Hiển thị giá trị của bộ đếm trong content.

 Ví dụ chi tiết:

 Bài toán: Tạo danh sách chương tự đánh số

```
<style>
  body {
```

```

        counter-reset: chapter; /* Bắt đầu bộ đếm tên
'chapter' từ 0 */
    }

    h2::before {
        counter-increment: chapter; /* Mỗi h2 thì
chapter tăng 1 */
        content: "Chương " counter(chapter) ": ";
    }
</style>

<h2>Giới thiệu</h2>
<h2>Thiết kế hệ thống</h2>
<h2>Triển khai</h2>

```

□ Kết quả trên trình duyệt:

Chương 1: Giới thiệu

Chương 2: Thiết kế hệ thống

Chương 3: Triển khai

flex-wrap

♦ Các giá trị của flex-wrap:

Giá trị	Ý nghĩa
nowrap	(mặc định) Không cho xuống dòng. Tất cả các item nằm trên một dòng.
wrap	Cho phép xuống dòng. Các item tràn ra sẽ tự động nhảy xuống dòng mới.

wrap-reverse	Giống wrap nhưng dòng mới xuất hiện phía trên dòng trước.
--------------	--

aspect-ratio

aspect-ratio là một **thuộc tính CSS** dùng để **đặt tỉ lệ chiều rộng so với chiều cao** của một phần tử — giống như nói "hình chữ nhật này có tỉ lệ 16:9" hay "1:1" (vuông).

-
- ♦ Cú pháp:

```
selector {  
    aspect-ratio: width / height;  
}
```

border-radius

border-radius là một **thuộc tính CSS** dùng để **bo tròn góc** của một phần tử HTML. Bạn có thể bo tròn nhẹ hoặc làm thành hình tròn hoàn toàn tùy vào giá trị.

-
- ♦ Cú pháp cơ bản:

```
selector {  
    border-radius: 10px;  
}
```

→ Làm cho **tất cả 4 góc** được bo tròn với bán kính 10px.

- ♦ Một số ví dụ:

```
/* Bo tròn đều 4 góc */
```

```
.box1 {  
  border-radius: 20px;  
}
```

```
/* Bo tròn từng góc riêng */
```

```
.box2 {  
  border-top-left-radius: 10px;  
  border-top-right-radius: 0;  
  border-bottom-right-radius: 30px;  
  border-bottom-left-radius: 5px;  
}
```

```
/* Làm hình tròn hoàn toàn (nếu là hình vuông) */
```

```
.circle {  
  width: 100px;  
  height: 100px;  
  border-radius: 50%;  
  background: pink;  
}
```

-
- ♦ Ứng dụng thực tế:

- Làm nút bo tròn đẹp hơn.
- Làm avatar hình tròn.
- Tạo khung mềm mại, dễ nhìn cho UI.

gap

gap là một **thuộc tính CSS** dùng để tạo **khoảng cách giữa các phần tử con** bên trong một **container** có `display: flex` hoặc `display: grid`.

- ♦ Cú pháp:

```
.container {  
  display: flex; /* hoặc grid */  
  gap: 20px;      /* tạo khoảng cách 20px giữa các  
phần tử con */  
}
```

- ♦ Cách hoạt động:

- **Không thêm khoảng cách bên ngoài container**, chỉ giữa các phần tử con.
- **Không cần margin từng phần tử** như trước kia (giúp code gọn hơn).

Table

1. Cơ bản

```
table {  
  border-collapse: collapse; /* Gộp đường viền */  
  width: 100%; /* Hoặc đặt theo px */  
}
```

```
th, td {  
    border: 1px solid #ddd;  
    padding: 8px;  
}
```

2. Căn giữa và định dạng text

```
th {  
    text-align: center;  
    background-color: #f2f2f2;  
}
```

```
td {  
    text-align: left;  
}
```

3. Hiệu ứng hover

```
tr:hover {  
    background-color: #f5f5f5;  
}
```

4. Zebra stripes (dòng xen kẽ màu)

```
tr:nth-child(even) {  
    background-color: #f9f9f9;  
}
```

5. Tạo bảng responsive (co giãn màn hình)

```
.table-container {  
    overflow-x: auto;  
}
```

HTML:

```
<div class="table-container">  
    <table>...</table>  
</div>
```

Background-image

1. background-image

Dùng để đặt hình ảnh nền.

```
background-image: url('path/to/image.jpg');
```

✓ 2. background-repeat

Điều khiển việc lặp lại hình nền.

```
background-repeat: no-repeat; /* không lặp */  
background-repeat: repeat; /* lặp cả 2 chiều (mặc định) */  
background-repeat: repeat-x; /* lặp theo chiều ngang */  
background-repeat: repeat-y; /* lặp theo chiều dọc */
```

✓ 3. background-size

Điều chỉnh kích thước hình nền.

```
background-size: cover; /* phủ kín toàn bộ phần tử, có thể bị cắt */  
background-size: contain; /* toàn bộ ảnh hiển thị, không bị cắt */  
background-size: 100% 100%; /* giãn ảnh theo chiều ngang & dọc */  
background-size: auto; /* kích thước gốc */
```

✓ 4. background-position

Xác định vị trí ảnh nền trong phần tử.

background-position: center;	/* chính giữa */
background-position: top left;	/* góc trên trái */
background-position: 50% 50%;	/* giữa ngang - giữa dọc */
background-position: 20px 40px;	/* cách trái 20px, trên 40px */

✓ 5. background-attachment

Quy định cách ảnh nền cuộn theo trang.

background-attachment: scroll;	/* cuộn theo nội dung (mặc định) */
background-attachment: fixed;	/* cố định trên màn hình */
background-attachment: local;	/* cuộn cùng phần tử chứa */

✓ 6. background-color

Màu nền (có thể dùng cùng background-image để làm nền phụ).

background-color: #f5f5f5

✓ 7. background (thu gọn)

Gộp tất cả vào một dòng:

css

background: url('image.jpg') no-repeat center center / cover #222 fixed;

Ý nghĩa thứ tự:

background: [image] [repeat] [position] / [size] [color] [attachment];

data-*

✓ Định nghĩa:

Thuộc tính `data-*` cho phép bạn lưu **dữ liệu tùy chỉnh** trên các phần tử HTML mà không ảnh hưởng đến cú pháp HTML hợp lệ.

✓ Cú pháp:

```
<div id="user" data-user-id="123" data-role="admin">User Info</div>
```

✓ Truy cập trong JavaScript:

```
const div = document.getElementById('user');
console.log(div.dataset.userId); // "123"
console.log(div.dataset.role);   // "admin"
```

- ♦ Các thuộc tính `data-` được lưu trong đối tượng `dataset`, và tên thuộc tính sẽ được chuyển thành `camelCase` (quy tắc con lạch đà)



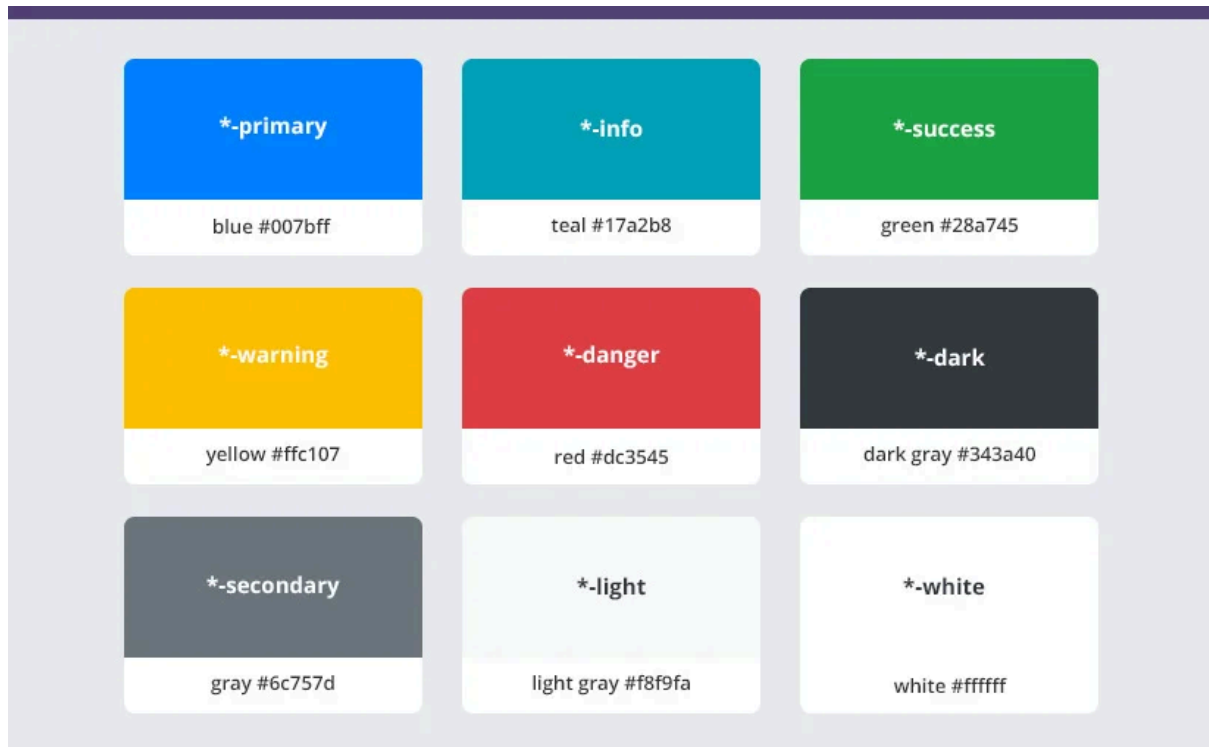
opacity là gì?

`opacity` là thuộc tính CSS dùng để điều chỉnh **độ trong suốt (độ mờ)** của một phần tử trên trang web.

- **Giá trị từ 0 đến 1:**
 - 1 → hoàn toàn **không trong suốt** (hiển thị rõ 100%)
 - 0 → hoàn toàn **trong suốt** (ẩn luôn, không thấy)
 - 0.5 → hiển thị **mờ 50%**

Bootstrap

Color



This text is muted.

This text is important.

This text indicates success.

This text represents some information.

This text represents a warning.

This text represents danger.

Secondary text.

Dark grey text.

Body text.

Light grey text.

Grid

```
<div class="row">
  <div class="col-*-*"></div>
  <div class="col-*-*"></div>
</div>
<div class="row">
  <div class="col-*-*"></div>
  <div class="col-*-*"></div>
  <div class="col-*-*"></div>
</div>
```

```
<!-- Or let Bootstrap automatically handle the layout -->
<div class="row">
  <div class="col"></div>
  <div class="col"></div>
  <div class="col"></div>
</div>
```

Display

```
<h1 class="display-1">Display 1</h1>
<h1 class="display-2">Display 2</h1>
<h1 class="display-3">Display 3</h1>
<h1 class="display-4">Display 4</h1>
<h1 class="display-5">Display 5</h1>
<h1 class="display-6">Display 6</h1>
```

Typography

<small>, <mark>
<abbr>: gạch chấm dưới chân
<blockquote>

<pre><dl> <dt>Coffee</dt> <dd>- black hot drink</dd> <dt>Milk</dt> <dd>- white cold drink</dd> </dl></pre>	Coffee - black hot drink Milk - white cold drink
--	---

Table

table-striped (không viền chỉ có viền ngăn cách giữa các dòng),
table-bordered, table-hover, table-borderless (không viền)

Image

Image shaped: `.rounded`, `.rounded-circle`, `.img-thumbnail`

Aligning Images

Float an image to the left with the `.float-start` class or to the right with `.float-end`:

Centered Image

Center an image by adding the utility classes `.mx-auto` (`margin:auto`) and `.d-block` (`display:block`) to the image:

```

```

Alert

```
div class="alert alert-success">
  <strong>Success!</strong> This alert box could indicate a successful
  or positive action.
</div>
```

Success! This alert box could indicate a successful or positive action.

Button

Style:

```
<button type="button" class="btn
btn-primary">Primary</button>
```

Outline

```
<button type="button" class="btn
btn-outline-primary">Primary</button>
```

Sizes

```
<button type="button" class="btn btn-primary  
btn-lg">Large</button>  
<button type="button" class="btn btn-primary">Default</button>  
<button type="button" class="btn btn-primary  
btn-sm">Small</button>
```

Active: makes a button appear pressed

```
<button type="button" class="btn btn-primary  
active">Active Primary</button>
```

Disabled: makes a button unclickable

```
<a href="#" class="btn btn-primary disabled">Disabled  
Link</a>
```

Button-group (inline)

Ngang

```
<div class="btn-group">  
  <button type="button" class="btn  
btn-primary">Apple</button>  
  <button type="button" class="btn  
btn-primary">Samsung</button>  
  <button type="button" class="btn  
btn-primary">Sony</button>  
</div>
```

Dọc

```
<div class="btn-group-vertical">  
  <button type="button" class="btn  
btn-primary">Apple</button>  
  <button type="button" class="btn  
btn-primary">Samsung</button>  
  <button type="button" class="btn  
btn-primary">Sony</button>  
</div>
```

Lớn, nhỏ: btn-group-lg, btn-group-sm

Có drop-down

```
<div class="btn-group">  
  <button type="button" class="btn  
btn-primary">Apple</button>
```

```

    <button type="button" class="btn
btn-primary">Samsung</button>
    <div class="btn-group">
        <button type="button" class="btn btn-primary
dropdown-toggle"
data-bs-toggle="dropdown">Sony</button>
        <div class="dropdown-menu">
            <a class="dropdown-item" href="#">Tablet</a>
            <a class="dropdown-item" href="#">Smartphone</a>
        </div>
    </div>
</div>

```

Progress bar

```

<div class="progress" style="height:10px">
    <div class="progress-bar bg-success"
style="width:40%;"></div>
</div>
<br>
<div class="progress" style="height:20px">
    <div class="progress-bar" style="width:50%;"></div>
</div>
<br>
<div class="progress" style="height:30px">
    <div class="progress-bar" style="width:60%;"></div>
</div>

```

Spinners

spinner border

```
<div class="spinner-border text-primary"></div>
```

spinner grow

```
<div class="spinner-grow text-muted"></div>
```

spinner + button:

```
<button class="btn btn-primary">
  <span class="spinner-border
spinner-border-sm"></span>
</button>
```

Pagination (chuyển trang)

```
<ul class="pagination">
  <li class="page-item"><a class="page-link"
href="#">Previous</a></li>
  <li class="page-item"><a class="page-link"
href="#">1</a></li>
  <li class="page-item active"><a class="page-link"
href="#">2</a></li>
  <li class="page-item"><a class="page-link"
href="#">3</a></li>
  <li class="page-item"><a class="page-link"
href="#">Next</a></li>
</ul>
```

active để đánh dấu trang hiện tại

List group

Active item
Second item
Third item

```
<ul class="list-group">
  <li class="list-group-item active">Active
item</li>
  <li class="list-group-item">Second item</li>
  <li class="list-group-item">Third item</li>
</ul>
```

First item

Second item

Third item

Fourth item

```
<ul class="list-group list-group-flush">
  <li class="list-group-item">First item</li>
  <li class="list-group-item">Second item</li>
  <li class="list-group-item">Third item</li>
  <li class="list-group-item">Fourth item</li>
</ul>
```

(không border)

1. First item

2. Second item

3. Third item

```
<ol class="list-group list-group-numbered">
  <li class="list-group-item">First item</li>
  <li class="list-group-item">Second item</li>
  <li class="list-group-item">Third item</li>
</ol>
```

First item

Second item

Third item

Fourth item

```
<ul class="list-group list-group-horizontal">
  <li class="list-group-item">First item</li>
  <li class="list-group-item">Second item</li>
  <li class="list-group-item">Third item</li>
  <li class="list-group-item">Fourth item</li>
</ul>
```

Badge

Vuông viền tròn

```
<span class="badge bg-primary">Primary</span>
```

Tròn hơn với rounded-pill

```
<span class="badge rounded-pill bg-primary">Primary</span>
```

Badge trong element

```
<button type="button" class="btn btn-primary">  
  Messages <span class="badge bg-danger">4</span>  
</button>
```

Data-bs-toggle

data-bs-toggle là một attribute (thuộc tính) trong Bootstrap (từ phiên bản 5 trở đi), dùng để kích hoạt các thành phần tương tác như modal, dropdown, tab, collapse... mà không cần viết JavaScript thủ công.

Bootstrap sử dụng data-bs-toggle để hiểu **kiểu chức năng tương tác** mà bạn muốn áp dụng lên phần tử. Dưới đây là một số giá trị thường dùng:

Giá trị của data-bs-toggle	Ý nghĩa	Ví dụ kết hợp
modal	Mở hộp thoại (modal)	data-bs-target="#modalId"
collapse	Mở/đóng khối nội dung ẩn	data-bs-target="#collapseId"
dropdown	Hiển thị menu xổ xuống	Gắn với class .dropdown-menu
tab	Chuyển tab	Gắn với các nav-link/tab-pane
offcanvas	Mở thanh trượt bên (offcanvas)	data-bs-target="#offcanvasId"

tooltip	Hiển thị tooltip (kết hợp với data-bs-placement)	Dùng chung với JavaScript
---------	--	---------------------------

Dropdown

```
<div class="dropdown">
  <button type="button" class="btn btn-primary
dropdown-toggle" data-bs-toggle="dropdown">
    Dropdown button
  </button>
  <ul class="dropdown-menu">
    <li><a class="dropdown-item" href="#">Link 1</a></li>
    <li><a class="dropdown-item" href="#">Link 2</a></li>
    <li><a class="dropdown-item" href="#">Link 3</a></li>
  </ul>
</div>
```

Dropright

```
<div class="dropdown dropend">
```

Dropleft

```
<div class="dropdown dropstart">
```

Collapse

```
<p class="d-inline-flex gap-1">
  <a class="btn btn-primary" data-bs-toggle="collapse"
href="#collapseExample" role="button"
aria-expanded="false" aria-controls="collapseExample">
    Link with href
  </a>
  <button class="btn btn-primary" type="button"
data-bs-toggle="collapse"
data-bs-target="#collapseExample" aria-expanded="false"
aria-controls="collapseExample">
    Button with data-bs-target
```

```
        </button>
</p>
<div class="collapse" id="collapseExample">
  <div class="card card-body">
    Some placeholder content for the collapse
    component. This panel is hidden by default but revealed
    when the user activates the relevant trigger.
  </div>
</div>
```

aria-expanded

✓ Mục đích:

Dùng để thông báo cho **các công cụ hỗ trợ (như trình đọc màn hình)** biết rằng một phần tử (ví dụ: dropdown, menu, accordion...) đang **mở (expanded)** hay **đóng (collapsed)**.

📌 Giá trị:

- true: phần tử đang mở.
- false: phần tử đang đóng.

aria-controls

✓ Mục đích:

Xác định **ID của phần tử mà nút này điều khiển**. Điều này giúp các công cụ hỗ trợ hiểu được **nút này đang tương tác với vùng nội dung nào**.

📌 Giá trị:

- Là **ID của phần tử mục tiêu** (được điều khiển).

margin

Cú pháp chung:

`m{side}-{size}`

Trong đó:

Phần	Ý nghĩa
m	Margin
{side}	Hướng áp dụng (xem bảng bên dưới)
{size}	Kích thước từ 0 đến 5, hoặc auto

 Bảng hướng (sides):

Tên	Ý nghĩa	Hướng áp dụng
t	top	Trên
b	bottom	Dưới
s	start	Bên trái (trong LTR)
e	end	Bên phải (trong LTR)
x	trái + phải	
y	trên + dưới	
(rỗng)	tất cả 4 hướng	

Forms

```
<form action="/action_page.php">  
  <div class="mb-3 mt-3">
```

```

        <label for="email" class="form-label">Email:</label>
        <input type="email" class="form-control" id="email"
placeholder="Enter email" name="email">
    </div>
    <div class="mb-3">
        <label for="pwd" class="form-label">Password:</label>
        <input type="password" class="form-control" id="pwd"
placeholder="Enter password" name="pswd">
    </div>
    <div class="form-check mb-3">
        <label class="form-check-label">
            <input class="form-check-input" type="checkbox"
name="remember"> Remember me
        </label>
    </div>
    <button type="submit" class="btn
btn-primary">Submit</button>
</form>

```

Comment:

```

<label for="comment">Comments:</label>
<textarea class="form-control" rows="5" id="comment"
name="text"></textarea>

```

Input size:

```

<input type="text" class="form-control form-control-lg"
placeholder="Large input">
<input type="text" class="form-control"
placeholder="Normal input">
<input type="text" class="form-control form-control-sm"
placeholder="Small input">

```

Check

```

<div class="form-check">
    <input class="form-check-input" type="checkbox"
id="check1" name="option1" value="something" checked>
    <label class="form-check-label">Option 1</label>
</div>

```

Radio

```
<div class="form-check">
  <input type="radio" class="form-check-input"
id="radio1" name="optradio" value="option1"
checked>Option 1
  <label class="form-check-label" for="radio1"></label>
</div>
<div class="form-check">
  <input type="radio" class="form-check-input"
id="radio2" name="optradio" value="option2">Option 2
  <label class="form-check-label" for="radio2"></label>
</div>
<div class="form-check">
  <input type="radio" class="form-check-input"
disabled>Option 3
  <label class="form-check-label"></label>
</div>
```

Toggle Switches

```
<div class="form-check form-switch">
  <input class="form-check-input" type="checkbox"
id="mySwitch" name="darkmode" value="yes" checked>
  <label class="form-check-label" for="mySwitch">Dark
Mode</label>
</div>
```

Card

```
<div class="card" style="width: 18rem;">
  
  <div class="card-body">
    <h5 class="card-title">Card title</h5>
    <p class="card-text">Some quick example text to build on the card
title and make up the bulk of the card's content.</p>
    <a href="#" class="btn btn-primary">Go somewhere</a>
  </div>
</div>
```

JS

element.classList

Là một **đối tượng** chứa danh sách các class trên phần tử HTML. Bạn có thể:

- Thêm class
- Xóa class
- Kiểm tra class
- Chuyển đổi (thêm nếu chưa có, xóa nếu có)
- Và nhiều thao tác khác

Các phương thức phổ biến của classList:

Phương thức	Mô tả	Ví dụ
add(className 1, ...)	Thêm một hoặc nhiều class vào phần tử	<code>el.classList.add("active", "red")</code>
remove(className1, ...)	Xóa một hoặc nhiều class khỏi phần tử	<code>el.classList.remove("hidden")</code>
toggle(className, [force])	Thêm nếu chưa có, xóa nếu có. Nếu truyền force (true/false) thì điều khiển chắc chắn thêm hay xóa	<code>el.classList.toggle("active")</code> <code>el.classList.toggle("active", true)</code> để luôn thêm
contains(className)	Kiểm tra phần tử có class đó hay không	<code>el.classList.contains("show")</code> → true hoặc false

replace(oldClass, newClass)	Thay thế class cũ bằng class mới	<code>el.classList.replace("old", "new")</code>
item(index)	Trả về tên class ở vị trí index (theo thứ tự trong danh sách class)	<code>el.classList.item(0)</code>

Ví dụ minh họa:

```
const el = document.querySelector("#myDiv");
```

```
// Thêm class
```

```
el.classList.add("active");
```

```
// Xóa class
```

```
el.classList.remove("hidden");
```

```
// Kiểm tra class
```

```
if (el.classList.contains("active")) {
```

```
    console.log("Phần tử đang active");
```

```
}
```

```
// Chuyển đổi class
```

```
el.classList.toggle("highlight");
```

```
// Thay thế class
```

```
el.classList.replace("oldClass", "newClass");
```

```
// Lấy class đầu tiên
```

```
console.log(el.classList.item(0));
```

🐭 chíp chíp~ Chuột sẽ giải thích rõ ràng cho bạn về **localStorage** – một trong những công cụ lưu trữ dữ liệu phổ biến nhất trong trình duyệt!

localStorage là gì?

localStorage là một dạng bộ nhớ cục bộ (web storage) dùng để lưu trữ dữ liệu dưới dạng chuỗi (string) trực tiếp trong trình duyệt của người dùng, không bị mất khi reload hay tắt trình duyệt.

✓ Đặc điểm chính:

Tính chất	Mô tả
Dung lượng	Khoảng 5MB tùy trình duyệt
Loại dữ liệu	Chỉ lưu được chuỗi (string)
Thời gian sống	Vĩnh viễn , trừ khi xóa bằng code hoặc tay
Phạm vi	Dữ liệu được lưu theo domain
Đồng bộ	Đồng bộ (synchronous) – có thể gây block nếu thao tác lớn

 Cú pháp sử dụng:

Hành động	Cú pháp
Lưu dữ liệu	<code>localStorage.setItem(key, value)</code>
Lấy dữ liệu	<code>localStorage.getItem(key)</code>
Xóa một mục	<code>localStorage.removeItem(key)</code>
Xóa toàn bộ	<code>localStorage.clear()</code>

Đếm số mục	<code>localStorage.length</code>
------------	----------------------------------

🧠 Ví dụ cơ bản:

// Lưu

```
localStorage.setItem('name', 'Chuột Chíp');
```

// Lấy

```
const name = localStorage.getItem('name'); // "Chuột Chíp"
```

// Xóa

```
localStorage.removeItem('name');
```

// Xóa hết

```
localStorage.clear();
```

💡 Lưu object hoặc mảng?

Vì chỉ lưu được chuỗi, nên phải dùng `JSON.stringify()` và `JSON.parse()`:

```
const book = { title: 'React', year: 2023 };
```

// Lưu

```
localStorage.setItem('book', JSON.stringify(book));
```

// Lấy lại

```
const stored = localStorage.getItem('book');
```

```
const parsedBook = JSON.parse(stored);
```

⚠️ Lưu ý khi dùng `localStorage`:

- Không nên lưu dữ liệu **nhạy cảm** (mật khẩu, token) vì nó **có thể bị xem dễ dàng** qua dev tools.

- Tránh lưu quá nhiều dữ liệu (giới hạn ~5MB).
- localStorage hoạt động **đồng bộ**, có thể gây chậm nếu thao tác với dữ liệu lớn.
- Có thể bị chặn nếu người dùng **dùng chế độ ẩn danh hoặc giới hạn cookie**.

Nếu bạn muốn chuột minh họa bằng ví dụ thực tế (form lưu dữ liệu người dùng, danh sách todo, trạng thái dark mode, v.v.), cứ nói nhé!



JQUERY

1. jQuery - Lý thuyết cơ bản

1.1. jQuery là gì?

- jQuery là thư viện JavaScript nhẹ, nhanh và dễ sử dụng, giúp thao tác với DOM, xử lý sự kiện, hiệu ứng và Ajax dễ dàng hơn.
- Viết code ngắn gọn, dễ đọc hơn so với JavaScript thuần.

1.2. Cách sử dụng jQuery

- Chèn thư viện jQuery vào trang HTML (thường là qua CDN).

```
<script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
```

- Cú pháp cơ bản: `$(selector).action()`

- Ví dụ: `$( '#id' ).hide();` — ẩn phần tử có id là id.

1.3. Selector (Chọn phần tử)

- Giống CSS selector, ví dụ:
 - `$( '#id' )` — chọn theo id
 - `$( '.class' )` — chọn theo class
 - `$( 'tag' )` — chọn theo thẻ HTML
 - Chọn phần tử con, anh em, cha mẹ, v.v.

1.4. DOM Manipulation (Thao tác DOM)

- Thêm, xóa, sửa nội dung, thuộc tính:
 - `.html(), .text(), .val()`
 - `.attr(), .removeAttr()`
 - `.append(), .prepend(), .remove(), .empty()`

1.5. Xử lý sự kiện (Event Handling)

- Đăng ký sự kiện: `.click(), .hover(), .on(), .off()`
- Ví dụ:

```
$( '#btn' ).click(function() {  
    alert('Clicked!');  
});
```

1.6. Hiệu ứng (Effects)

- Ẩn/hiện: `.hide()`, `.show()`, `.toggle()`
- Hiệu ứng mờ dần: `.fadeIn()`, `.fadeOut()`
- Hiệu ứng trượt: `.slideUp()`, `.slideDown()`

1.7. jQuery Chaining

- Gọi nhiều phương thức liên tiếp trên cùng một đối tượng:

```
$('#id').css('color', 'red').slideUp(2000).slideDown(2000);
```

1.8. Document Ready

- Đảm bảo DOM đã tải xong mới chạy code jQuery:

```
$(document).ready(function(){  
    // code ở đây  
});
```

- Viết tắt: `$(function(){ ... });`

AJAX

2. Ajax - Lý thuyết cơ bản

2.1. Ajax là gì?

- Ajax = Asynchronous JavaScript and XML.
- Kỹ thuật giúp gửi và nhận dữ liệu từ server mà không cần tải lại trang (load lại toàn bộ trang).

2.2. Các lợi ích của Ajax

- Cải thiện trải nghiệm người dùng
- Tải dữ liệu động
- Giao tiếp nhanh, mượt mà giữa client và server

2.3. Cách sử dụng Ajax với jQuery

2.3.1. \$.ajax()

- Hàm tổng quát dùng để gọi Ajax.

```
$.ajax({  
  url: 'server.php',  
  method: 'GET',  
  data: { id: 123 },  
  success: function(response) {  
    console.log(response);  
  },  
  error: function(xhr, status, error) {  
    console.error(error);  
  }  
});
```

2.3.2. Các phương thức tiện ích khác

- `.get()`: Gửi GET request
- `.post()`: Gửi POST request
- `.getJSON()`: Gửi GET request nhận dữ liệu JSON

2.4. Các tham số quan trọng của Ajax

- `url`: Đường dẫn server nhận request
- `method / type`: GET, POST, PUT, DELETE...
- `data`: Dữ liệu gửi đi (đối tượng, chuỗi query...)
- `dataType`: Dạng dữ liệu mong muốn nhận về (json, xml, html, text)
- `success`: Callback khi request thành công
- `error`: Callback khi request lỗi

2.5. Xử lý dữ liệu trả về

- Có thể dùng để cập nhật giao diện mà không cần reload trang
- Thường nhận dữ liệu JSON hoặc HTML từ server

2.6. Bất đồng bộ (Asynchronous)

- Mặc định Ajax chạy bất đồng bộ, không làm đứng trang web
- Có thể tùy chọn chạy đồng bộ (`async: false`) nhưng không khuyến khích

3. Tổng kết

Chủ đề	Nội dung chính
jQuery	Selector, DOM Manipulation, Event, Effects, Chaining, Document Ready
Ajax	Gửi nhận dữ liệu không tải lại trang, \$.ajax(), get(), post(), xử lý success/error, async

Yup

1. Kiểm tra với string() – Chuỗi

Kiểm tra	Ý nghĩa	Ví dụ
<code>.required(msg)</code>	Bắt buộc nhập	<code>Yup.string().required('Không được để trống')</code>
<code>.min(n, msg)</code>	Độ dài tối thiểu	<code>.min(6, 'Tối thiểu 6 ký tự')</code>
<code>.max(n, msg)</code>	Độ dài tối đa	<code>.max(20, 'Không quá 20 ký tự')</code>
<code>.email(msg)</code>	Định dạng email	<code>.email('Email sai định dạng')</code>
<code>.matches(regex, msg)</code>	Khớp với Regex	<code>.matches(/^[0-9]+\$/, 'Chỉ được nhập số')</code>
<code>.oneOf([...], msg)</code>	Phải thuộc tập giá trị	<code>.oneOf(['admin', 'user'], 'Loại tài khoản không hợp lệ')</code>

<code>.url(msg)</code>	Kiểm tra URL	<code>.url('URL không hợp lệ')</code>
------------------------	--------------	---------------------------------------

2. Kiểm tra với `number()` – Số

Kiểm tra	Ý nghĩa	Ví dụ
<code>.required(msg)</code>	Bắt buộc nhập	<code>.required('Phải nhập số')</code>
<code>.typeError(msg)</code>	Không đúng kiểu số	<code>.typeError('Phải là số')</code>
<code>.positive(msg)</code>	> 0	<code>.positive('Phải lớn hơn 0')</code>
<code>.negative(msg)</code>	< 0	<code>.negative('Phải nhỏ hơn 0')</code>
<code>.min(n, msg)</code>	$\geq n$	<code>.min(18, 'Ít nhất 18')</code>
<code>.max(n, msg)</code>	$\leq n$	<code>.max(65, 'Không quá 65')</code>
<code>.integer(msg)</code>	Phải là số nguyên	<code>.integer('Phải là số nguyên')</code>

3. Kiểm tra với `date()` – Ngày tháng

Kiểm tra	Ý nghĩa	Ví dụ
----------	---------	-------

<code>.required(msg)</code>	Bắt buộc nhập	<code>.required('Chọn ngày')</code>
<code>.min(date, msg)</code>	$\geq$ ngày	<code>.min(new Date(), 'Không được chọn ngày trong quá khứ')</code>
<code>.max(date, msg)</code>	$\leq$ ngày	<code>.max(new Date('2025-12-31'), 'Quá hạn')</code>